

EXPERIMENT-10

Student Name: Husanpreet Kaur
Branch: AIT-CSE (Cyber Security)
Semester: 5
Subject Name: Computer Networks

UID: 24BCY70138
Section/Group: 24AIT_KRG-G2
Date: 23/09/2026
Subject Code: 24CSH-337

AIM: To capture and analyse network packets using Wireshark Packet Analyzer and understand the structure and communication of different network protocols such as ICMP, ARP, DNS, and TCP. To implement a client-server communication system using Python UDP sockets over the IPv4 loopback interface, and to demonstrate connectionless message transmission, server responses, and termination of communication using a predefined message.

OBJECTIVES:

- To understand the basic functionality of Wireshark as a network packet analyzer.
- To capture and analyse ICMP packets generated using the ping command.
- To capture and analyse ARP request and reply packets.
- To capture and analyse DNS query and response packets.
- To capture and analyse TCP connection packets and identify the TCP three-way handshake.

SOFTWARE REQUIREMENTS:

(a) Software:

- Wireshark
- Windows Operating System
- Command Prompt (CMD)
- Active Ethernet/Wi-Fi/mobile hotspot connection
- Npcap for packet capture on Windows

THEORY: (i) What is Wireshark?

EXPERIMENT-10

Wireshark is a network packet analyzer used to capture and inspect network traffic. It can capture packets from an active network interface and display protocol-level information.

Wireshark uses the network interface and packet-capture tools such as Npcap on Windows to obtain raw packets from the operating system.

(ii) What is Promiscuous Mode?

Normally, a network interface processes packets intended for the device. Wireshark can enable Promiscuous Mode to capture packets available on the network segment, subject to the capabilities and configuration of the network interface.

(iii) What is Monitor Mode?

For wireless interfaces, **Monitor Mode** can capture wireless frames in the air, including frames not specifically addressed to the device, depending on hardware and driver support.

(iv) Packet Capture Flow:

Network Traffic → Network Interface → Npcap → Operating System → Wireshark → Packet Analysis

PROTOCOLS ANALYSED:

- HTTP / HTTPS
- ARP
- DHCP
- DNS
- ICMP
- SSH / FTP
- TCP

The experiment specifically demonstrates ICMP, ARP, DNS, and TCP packet capture.

EXPERIMENT-10

STEPS/IMPLEMENTATION:

EXPERIMENT 1: ICMP PACKET CAPTURE

Procedure:

1. Connect the laptop to the Internet using Ethernet, Wi-Fi, or mobile hotspot.
2. Open Wireshark.
3. Select the active network interface.
4. Double-click the interface to start packet capture.
5. Open Command Prompt.
6. Execute: ping 8.8.8.8
7. Wait for the ping command to finish.
8. Stop packet capture.
9. Apply the filter: icmp && ip.addr == 8.8.8.8
10. Identify the Echo Request and corresponding Echo Reply packets.

Packet Size Analysis:

Component	Size
Ethernet Header	14 bytes
IPv4 Header	20 bytes
ICMP Header	8 bytes
Ping Data	32 bytes
Total	74 bytes

EXPERIMENT 2: ARP PACKET CAPTURE

Procedure:

EXPERIMENT-10

1. Connect the laptop to Ethernet/Wi-Fi/mobile hotspot.
2. Open Command Prompt.
3. Execute: ipconfig
4. Note the Default Gateway IP.
5. Open Command Prompt as Administrator.
6. Clear the dynamic ARP cache: arp -d *
7. Open Wireshark.
8. Select the active network interface.
9. Start packet capture.
10. Execute: ping <Default-Gateway-IP>
Eg: ping 192.168.0.1
11. Stop packet capture.
12. Apply the filter: arp
13. Identify the ARP Request and corresponding ARP Reply.
14. Analyse:
 - Source
 - Destination
 - Length
 - Info
 - Sender IP/MAC
 - Target IP/MAC

EXPERIMENT 3: DNS PACKET CAPTURE

Procedure:

EXPERIMENT-10

1. Connect the laptop to the Internet.
2. Open Wireshark.
3. Select the active Ethernet or Wi-Fi interface.
4. Start packet capture.
5. Open Command Prompt.
6. Execute: nslookup example.com
7. Wait for the DNS result.
8. Return to Wireshark.
9. Stop packet capture.
10. Apply the filter: dns.qry.name == "example.com"
11. Identify the DNS query and response.
12. Analyse:
 - Source
 - Destination
 - Protocol
 - Length
 - Info
13. Inspect the DNS and UDP fields.

EXPERIMENT 4: TCP PACKET CAPTURE

Procedure:

1. Connect the laptop to the Internet.
2. Open Wireshark.
3. Select the active interface.

EXPERIMENT-10

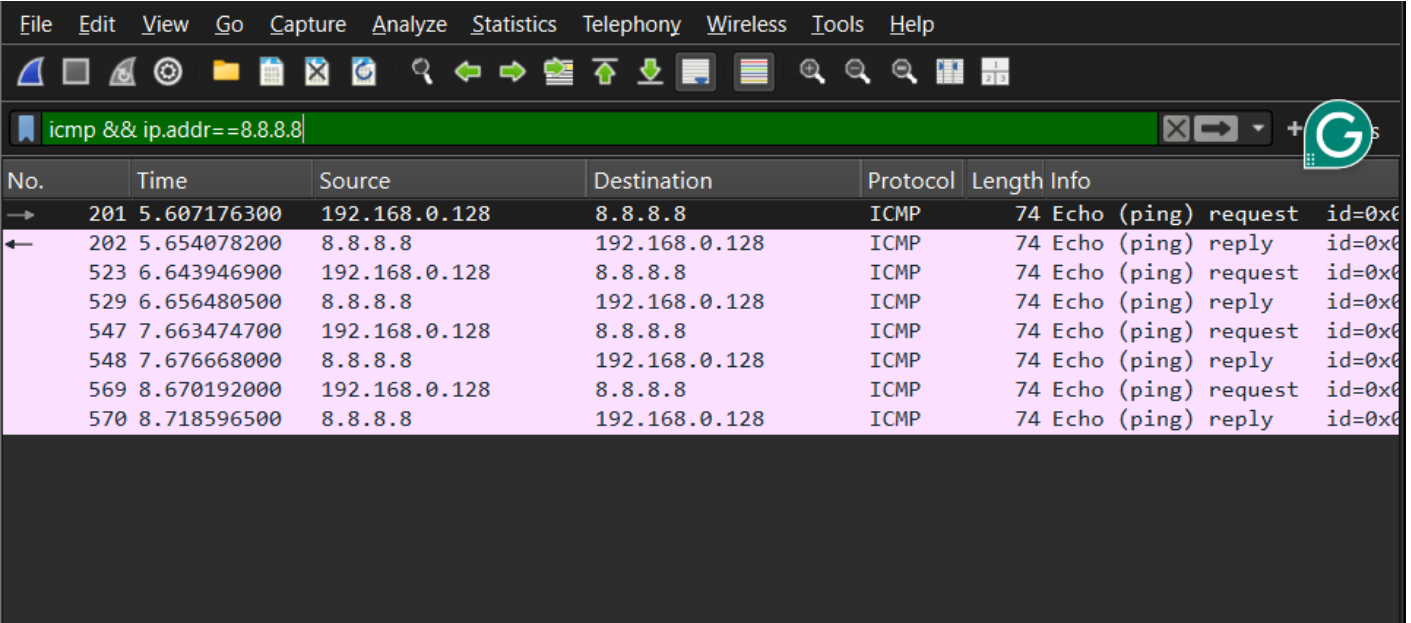
4. Start packet capture.
5. Open Command Prompt.
6. Execute: curl http://example.com
7. Wait for the command to complete.
8. Return to Wireshark.
9. Stop packet capture.
10. Apply the filter: tcp.port == 80
11. Identify the TCP connection.
12. Observe the TCP three-way handshake:
[SYN]
↓
[SYN, ACK]
↓
[ACK]
13. Examine:
 - Source
 - Destination
 - Source Port
 - Destination Port
 - Sequence Number
 - Acknowledgment Number
 - TCP Flags
14. Optionally use Follow → TCP Stream to inspect the HTTP communication.

EXPERIMENT-10

IMPORTANT WIRESHARK FILTERS

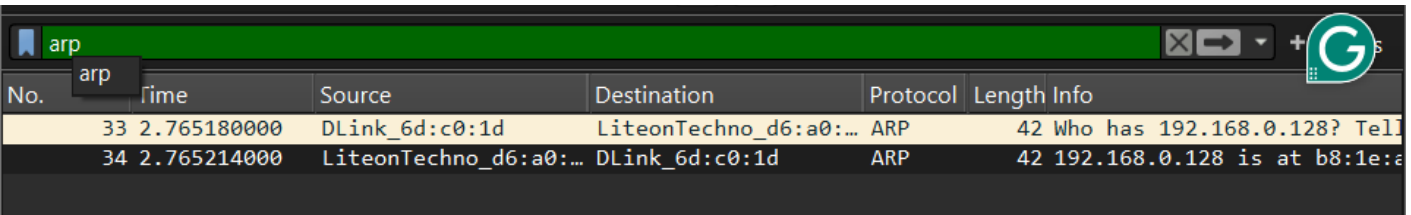
Protocol	Wireshark Filter
ICMP	icmp
ICMP with specific IP	icmp && ip.addr == 8.8.8.8
ARP	arp
DNS	dns
Specific DNS query	dns.qry.name == "example.com"
TCP Port 80	tcp.port == 80

DEMONSTRATION/OUTPUT:



Wireshark capture showing ICMP ping requests and replies. The filter applied is `icmp && ip.addr == 8.8.8.8`. The capture shows a sequence of requests and replies between the source IP 192.168.0.128 and the destination IP 8.8.8.8.

No.	Time	Source	Destination	Protocol	Length	Info
201	5.607176300	192.168.0.128	8.8.8.8	ICMP	74	Echo (ping) request id=0x0
202	5.654078200	8.8.8.8	192.168.0.128	ICMP	74	Echo (ping) reply id=0x0
523	6.643946900	192.168.0.128	8.8.8.8	ICMP	74	Echo (ping) request id=0x0
529	6.656480500	8.8.8.8	192.168.0.128	ICMP	74	Echo (ping) reply id=0x0
547	7.663474700	192.168.0.128	8.8.8.8	ICMP	74	Echo (ping) request id=0x0
548	7.676668000	8.8.8.8	192.168.0.128	ICMP	74	Echo (ping) reply id=0x0
569	8.670192000	192.168.0.128	8.8.8.8	ICMP	74	Echo (ping) request id=0x0
570	8.718596500	8.8.8.8	192.168.0.128	ICMP	74	Echo (ping) reply id=0x0



Wireshark capture showing ARP requests and replies. The filter applied is `arp`. The capture shows an ARP request from DLink_6d:c0:1d to LiteonTechno_d6:a0:1d and a corresponding reply.

No.	Time	Source	Destination	Protocol	Length	Info
33	2.765180000	DLink_6d:c0:1d	LiteonTechno_d6:a0:1d	ARP	42	Who has 192.168.0.128? Tell...
34	2.765214000	LiteonTechno_d6:a0:1d	DLink_6d:c0:1d	ARP	42	192.168.0.128 is at b8:1e:...

EXPERIMENT-10

dns

No.	Time	Source	Destination	Protocol	Length	Info
21	3.5990459600	192.168.0.128	192.168.0.1	DNS	84	Standard query 0x0001 PTR 1
22	3.599269900	192.168.0.1	192.168.0.128	DNS	120	Standard query response 0x0
23	3.601712400	192.168.0.128	192.168.0.1	DNS	71	Standard query 0x0002 A exa
24	3.616369000	192.168.0.1	192.168.0.128	DNS	103	Standard query response 0x0
25	3.623694900	192.168.0.128	192.168.0.1	DNS	71	Standard query 0x0003 AAAA
26	3.684982400	192.168.0.1	192.168.0.128	DNS	99	Standard query response 0x0

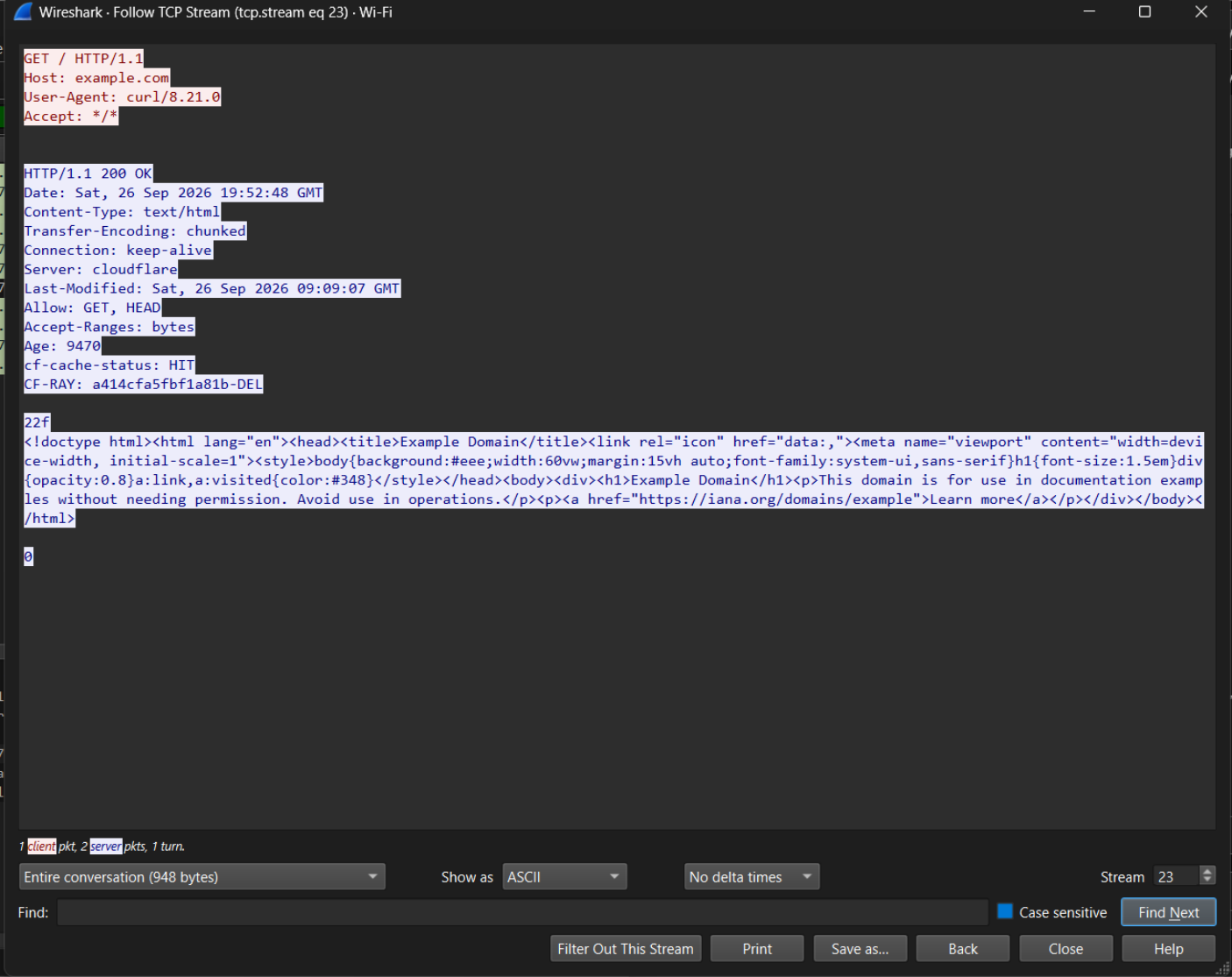
dns.qry.name=="example.com"

No.	Time	Source	Destination	Protocol	Length	Info
23	3.601712400	192.168.0.128	192.168.0.1	DNS	71	Standard query 0x0002 A exa
24	3.616369000	192.168.0.1	192.168.0.128	DNS	103	Standard query response 0x0
25	3.623694900	192.168.0.128	192.168.0.1	DNS	71	Standard query 0x0003 AAAA
26	3.684982400	192.168.0.1	192.168.0.128	DNS	99	Standard query response 0x0

tcp.port==80

No.	Time	Source	Destination	Protocol	Length	Info
191	6.662968400	192.168.0.128	172.66.147.243	TCP	66	10903 → 80 [SYN] Seq=0 Win=
192	6.679322900	172.66.147.243	192.168.0.128	TCP	66	80 → 10903 [SYN, ACK] Seq=0
193	6.679499200	192.168.0.128	172.66.147.243	TCP	54	10903 → 80 [ACK] Seq=1 Ack=
194	6.679807800	192.168.0.128	172.66.147.243	HTTP	129	GET / HTTP/1.1
195	6.691171400	172.66.147.243	192.168.0.128	TCP	54	80 → 10903 [ACK] Seq=1 Ack=
196	6.699606100	172.66.147.243	192.168.0.128	TCP	922	80 → 10903 [PSH, ACK] Seq=1
197	6.699606100	172.66.147.243	192.168.0.128	HTTP	59	HTTP/1.1 200 OK (text/htm
198	6.699774300	192.168.0.128	172.66.147.243	TCP	54	10903 → 80 [ACK] Seq=76 Ack
199	6.700771300	192.168.0.128	172.66.147.243	TCP	54	10903 → 80 [FIN, ACK] Seq=7
200	6.716379500	172.66.147.243	192.168.0.128	TCP	54	80 → 10903 [FIN, ACK] Seq=8
201	6.716472500	192.168.0.128	172.66.147.243	TCP	54	10903 → 80 [ACK] Seq=77 Ack

EXPERIMENT-10



Wireshark · Follow TCP Stream (tcp.stream eq 23) · Wi-Fi

```
GET / HTTP/1.1
Host: example.com
User-Agent: curl/8.21.0
Accept: */*

HTTP/1.1 200 OK
Date: Sat, 26 Sep 2026 19:52:48 GMT
Content-Type: text/html
Transfer-Encoding: chunked
Connection: keep-alive
Server: cloudflare
Last-Modified: Sat, 26 Sep 2026 09:09:07 GMT
Allow: GET, HEAD
Accept-Ranges: bytes
Age: 9470
cf-cache-status: HIT
CF-RAY: a414cfa5fbf1a81b-DEL

22f
<!doctype html><html lang="en"><head><title>Example Domain</title><link rel="icon" href="data:,"><meta name="viewport" content="width=device-width, initial-scale=1"><style>body{background:#eee;width:60vw;margin:15vh auto;font-family:system-ui,sans-serif}h1{font-size:1.5em}div{opacity:0.8}a:link,a:visited{color:#348}</style></head><body><div><h1>Example Domain</h1><p>This domain is for use in documentation examples without needing permission. Avoid use in operations.</p><p><a href="https://iana.org/domains/example">Learn more</a></p></div></body></html>
```

1 client pkt, 2 server pkts, 1 turn.

Entire conversation (948 bytes) Show as ASCII No delta times Stream 23

Find: Case sensitive Find Next

Filter Out This Stream Print Save as... Back Close Help

RESULT: The network packets were successfully captured and analysed using Wireshark. ICMP Echo Request/Reply, ARP Request/Reply, DNS query/response, and TCP connection packets were identified and examined using appropriate Wireshark filters. The experiment provided practical understanding of packet structure, protocol fields, addressing, and network communication.

LEARNING OUTCOMES:

After completing this experiment:

EXPERIMENT-10

- I understood the basic working of Wireshark and network packet capture.
- I learned to capture and analyse ICMP, ARP, DNS, and TCP packets.
- I learned to apply appropriate Wireshark display filters for protocol analysis.
- I understood important packet fields such as source, destination, ports, addresses, flags, and protocol information.
- I developed practical skills in analysing real network traffic and understanding protocol-level communication.